

F***brain ISA

F*brain** is a reversible computation language using Fredkin-style conditional swaps. The **F** is for Fredkin (and the name reverses Brainf***).

State

- `tape[0..N-1]` — memory (program + data + garbage), 8-bit cells
- `IP` — instruction pointer
- `CL` — control locus (condition for F, target for J/S/G)
- `H0` — data head 0 (accumulator, where +/- act)
- `H1` — data head 1 (swap partner for x/F)

All pointers wrap modulo N (tape size).

Instructions

Op	Name	Semantics	Inverse
<	CL left	CL -= 1	>
>	CL right	CL += 1	<
{	H0 left	H0 -= 1	}
}	H0 right	H0 += 1	{
(	H1 left	H1 -= 1	)
)	H1 right	H1 += 1	(
+	increment	tape[H0] += 1	-
-	decrement	tape[H0] -= 1	+
x	swap	swap(tape[H0], tape[H1])	self
F	Fredkin	if tape[CL] 0: swap(tape[H0], tape[H1])	self
J	jump	swap(IP, tape[CL])	self
S	bridge	swap(tape[CL], tape[H0])	self
G	indirect	swap(CL, tape[H0])	self

All operations are reversible. The tape *is* the program (von Neumann architecture).

True Reversibility

Every F***brain computation can be run backwards without maintaining a history stack. The simulator's `r` command computes the previous state directly by:

1. Determining if we arrived via J (check if `tape[CL]` points to a J instruction)
2. Applying the inverse operation (or re-executing self-inverse ops like x, F, J, S, G)

This is **true reversibility** — the physics of the computation itself is reversible, not just recorded.

Notes

F (Fredkin gate): Conditional swap enables branching. Tests `tape[CL]`; swaps `tape[H0]` `tape[H1]` if nonzero.

J (jump): Swaps IP with tape contents, enabling subroutines and loops. Writes return address for reversibility.

S (bridge): Swaps tape contents between CL and H0 positions. Useful for data movement.

G (indirect): Swaps CL register with tape contents. Enables computed/indirect addressing for CL. Essential for loops that need a different garbage cell each iteration.

Garbage area: Loops require $O(r)$ garbage cells for r iterations. Each loopback J writes a return address to a fresh cell. This is the “history” that enables reverse execution (Bennett’s approach).

Example: DEC-until-zero loop

`G{{{x}}})F>J<F>+{{{(x-)}}}+GJ` (31 instructions)

Decrements `tape[40]` until zero, using `tape[55+]` as garbage cells.

Memory layout: - `tape[40]` = r (loop counter) - `tape[41]` = scratch (for F test) - `tape[42]` = exit address - `tape[43]` = continue address - `tape[44]` = garbage pointer - `tape[55+]` = garbage cells (return addresses)

Initial pointers: IP=1, H0=44, H1=41, CL=41

The loop uses G twice per iteration: 1. **G at start:** Restores CL from `tape[44]` (undoes previous G) 2. **G before J:** Loads garbage pointer into CL for loopback

Each iteration writes one return address, using exactly r garbage cells for r iterations.